

Macro-Micro Collaborative Learning for Logical Data Center Microservice Indicators Forecasting

Mohan Gao
gao_mohan@sjtu.edu.cn
School of Computer Science,
Shanghai Jiao Tong University
Shanghai, China

Zhemeng Yu
fish_meng@sjtu.edu.cn
School of Computer Science,
Shanghai Jiao Tong University
Shanghai, China

Yang Luo
floating-dream@sjtu.edu.cn
School of Computer Science,
Shanghai Jiao Tong University
Shanghai, China

Lintao Ma
lintao.mlt@antgroup.com
Ant Group
Shanghai, China

Yinbo Sun
yinbo.syb@antgroup.com
Ant Group
Shanghai, China

Yuchen Fang
arthurfyc@gmail.com
Ant Group
Shanghai, China

Xiaofeng Gao*
gao-xf@cs.sjtu.edu.cn
Shanghai Key Laboratory of Scalable
Computing and Systems, School of
Computer Science, Shanghai Jiao
Tong University
Shanghai, China

Abstract

As microservice architecture is evolving toward Logical Data Center (LDC), accurate forecasting of the microservices indicators can support reasonable resource allocation, thereby ensuring the availability and reliability of cloud service. From a macro perspective, due to the architecture hierarchy, microservices exhibit: 1) collaborative relationships derived from shared functionalities, 2) backup relationships between replicas, and 3) dynamic correlation driven by cooperation. From a micro perspective, there exist causal relationships among indicators within a microservice. That is, workload will first impact system consumption, such as CPU and memory usage, then affect service quality like system latency.

Based on these insights, we propose MaMiClif, a macro-micro collaborative learning framework for LDC microservice indicators forecasting. MaMiClif constructs Macro Graph and Micro Matrix to model the microservices dependencies and the causality of indicators. To learn fine-grained indicator dependencies, Indicator-Centric Embedding is leveraged to generate representations for indicator series. We use Heterogeneous Graph Convolution to update workload representations based on the Macro Graph, and adopt Causal Sparse Self-attention to integrate causal strength into the self-attention calculation, enabling a comprehensive exploration of dependencies among indicators. Experiments on two datasets, including LDC_MS, which was collected from the LDC system of Ant Group, demonstrate the effectiveness of MaMiClif.

*Xiaofeng Gao is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792125>

CCS Concepts

• **Computer systems organization** → *Cellular architectures.*

Keywords

Logical Data Center, Microservice, Forecasting, Deep Learning.

ACM Reference Format:

Mohan Gao, Zhemeng Yu, Yang Luo, Lintao Ma, Yinbo Sun, Yuchen Fang, and Xiaofeng Gao. 2026. Macro-Micro Collaborative Learning for Logical Data Center Microservice Indicators Forecasting. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792125>

1 Introduction

With the rapid development of cloud services, to meet the needs of resource flexibility and fast deployment, cloud applications have evolved from a monolithic architecture to a distributed one represented by microservices. Microservice decomposes applications into small components from the function angle. These components cooperate to complete services and can be developed, deployed, and updated independently [12, 24]. To improve system availability and achieve load balancing, microservice systems are evolving toward Logical Data Center (LDC), which is a typical cellular architecture. LDC is a finer-grained division from the data angle, dividing the system into zones, within which microservices are responsible for processing a part of the overall data.

In LDC, the main challenge is to meet the cloud Service Level Agreement (SLA), which requires stabilizing resource utilization and latency at the required target level. Violation of the SLA can result in serious losses. For example, a 7-hour violation can cause \$18 million in economic damages¹. To address this challenge, the

¹<https://aws.amazon.com/cn/message/12721/>

system dynamically allocates resources based on the estimation of future indicators, such as workload and CPU usage [27, 46]. Therefore, accurate forecasting of indicators is the key to optimizing resource allocation [1, 11, 19].

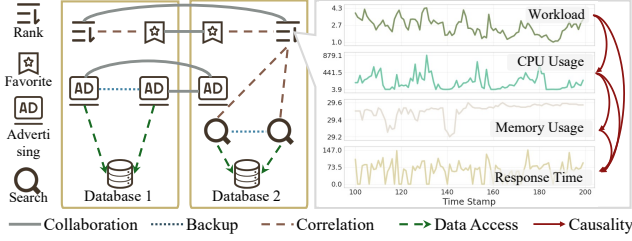


Figure 1: An instance of LDC Microservices, showing the complex but distinctive dependencies among microservices (left) and the microservice indicators causality (right).

As shown in Fig. 1, microservices in LDC exhibit complex dependencies. From a macro perspective, the system spans requests to the appropriate microservices based on the location of the relevant data. These microservices divide labor from the data angle to perform the same functionality, showing a collaborative relationship. To provide redundancy and respond to possible updates (details in Appx. A), there are microservice pairs with the same functionality and operating on the same data in the zones, showing a backup relationship. Furthermore, since the cooperative nature of microservices, there are also widespread correlation relationships among them due to functional interdependencies [13]. Recently, some workload forecasting methods based on Spatio-Temporal mining [13, 26] use indicator series similarity to characterize the relationship between microservices. These similarity-based methods mix relationships without making fine-grained distinctions, are easily affected by pseudo-correlations, and have poor robustness. From a micro perspective, causal relationships exist among the indicators within microservices [36]. Workload reflects the number of requests, and is the root cause of changes in other indicators that reflect system states. As illustrated in [27, 41], workload will directly affect consumption indicators like CPU and memory usage, which in turn affect latency like response time. A detailed discussion of these related work is provided in Appx. B.

Therefore, we propose the Macro-Micro Collaborative Learning for Logical Data Center Microservice Indicators Forecasting, called MaMiClif. First, Macro Graph and Micro Matrix are constructed to model the dependencies between microservices and the causality among indicators, respectively. The Macro Graph encodes architectural relationships—collaboration and backup—via heterogeneous edges, and supplements them with dynamic correlation edges derived from Pearson Correlation Coefficients. Meanwhile, the Micro Matrix quantifies causal strengths among indicators using Transfer Entropy. Second, Indicator-Centric Embedding is used to generate the representations of indicator series, while Heterogeneous Graph Convolution exploits the Macro Graph to learn the dependencies among microservice workloads. Finally, Causal Sparse Self-attention integrates causality into self-attention calculation, enabling a comprehensive exploration of dependencies among indicators, forming a macro-micro feature extraction structure.

Collectively, MaMiClif contains the insights of the LDC microservice system, including the heterogeneous architectural dependencies and the business correlation of microservices, as well as the extensive causal relationships among indicators, which are crucial for indicators forecasting. The contributions can be summarized as:

- We thoroughly investigate the LDC microservice system, precisely model the collaboration, backup, and correlation relationships among microservices. Macro graph is proposed to finely express these relationships.
- We adopt Micro Matrix to quantify the causal strength of indicators, and use Causal Sparse Self-attention to integrates causality into indicator correlation measurement.
- Extensive experiments on real-world datasets show that the proposed MaMiClif outperforms several state-of-the-art models by a prominent margin.

2 Problem Formulation

In this section, we will first clarify several key concepts, including microservice systems, logical data center hierarchy, and microservice indicator series, and then formally formulate the microservice indicator forecasting (MIF) problem.

2.1 Microservice System

In cloud service scenarios, a *service* refers to a program or a software that implements some particular business functionalities over one or multiple servers in the data center, running continuously in the system backend and processing requests. To improve scalability and efficiency, *microservices* are proposed as a means to decompose a service into multiple fine-grained, loosely coupled components [14], each of which can perform specific functionalities separately, or invoke a series of other components to form a service jointly.

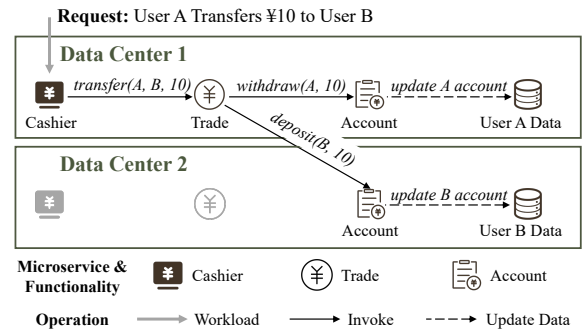


Figure 2: An example scenario of a money transfer service: User A transfers ¥10 to User B. The request is routed by the system to the Data Center 1, where User A's data is located.

Fig. 2 exhibits an example transfer service scenario: “User A transfers ¥10 to User B”, which involves 4 microservices corresponding to 3 functionalities. User A calls the Cashier microservice when the request comes, initiates a transfer through $transfer(A, B, 10)$. Since the data of User A and B are stored in different data centers, the Trade microservice invokes two Account microservices through

withdraw($A, 10$) and *deposit*($B, 10$) to complete the transfer. Finally, two Account microservices (corresponding to Users A and B) update the payer's and receiver's account data.

2.2 Logical Data Center Hierarchy

Logical Data Center (LDC) hierarchy divides computing resources of data centers into zones to guarantee service isolation, such that each independent LDC cell can operate autonomously, reducing mutual interference between different user groups or business domains. For instance, Ant Group operates large-scale data centers deployed across multiple locations, utilizes LDC to divide its computing resources into three tiers, labeled as DC (Data Center, refers to a physical data center); CG (Cell Group, refers to a collection of functionalities with microservice copies for backups and updates); and RZ (Region Zone, refers to the basic cell of such hierarchical partitioning), in a top-down manner.

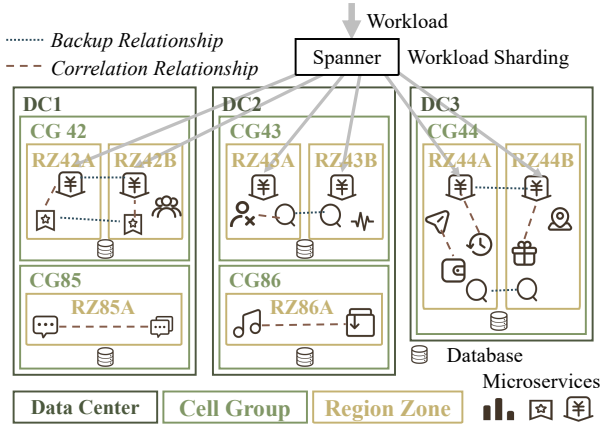


Figure 3: Logic Data Center Architecture. Microservices exhibit collaboration (via workload sharding), backup (within Cell Groups), and correlation (through cooperation).

Fig. 3 illustrates a top example with 3 Data Centers, 5 Cell Groups, and 8 Region Zones, each with a unique ID consisting of 1-2 digits. First, a Spanner implements the workload sharding mechanism to split requests into slices and assigns them to the corresponding RZs. Next, each RZ handles a slice independently through the cooperation of internal microservices, utilizing in-zone data, and will not escape to other zones. Cell Groups are designed with built-in redundancy. For instance, a pair of Trade microservices operates simultaneously in both RZ42A and RZ42B, with one serving as a backup and handling possible updates. The definition of microservices and its corresponding concepts are denoted as Def. 1.

DEFINITION 1 (MICROSERVICE). Define m as a microservice, and f_m as its functionality. dc_m , cg_m , and rz_m are labels representing the IDs of m 's Data Center, Cell Group, and Region Zone, respectively, that are deployed in the LDC. There are M microservices in this LDC.

2.3 Microservice Indicator Series

Containerization technology is adopted to deploy and execute microservices onto data centers. Container orchestration systems,

such as Docker [9] and Kubernetes [20], are used to monitor the *indicators* of the microservice system and achieve various advanced requirements, including efficient scheduling and auto-scaling [24, 45].

Indicators of a microservice include its workload and system states. The number of requests received by a microservice at a single point in time (time t) is referred to as *workload*. It reflects the trends of user demands and the company's business conditions [13], exhibits relatively high predictability, and serves as the core of all indicators. Other system state indicators, such as CPU usage (CPU cores $\times$ CPU utilization rate), memory usage (MU), and response time (RT, meaning the latency to reply to the request back to the users) reflect current resource consumption and service quality [27]. Combining multiple indicators, the definition of the indicator triple is given in Def. 2.

DEFINITION 2 (MICROSERVICE INDICATORS). Define the indicator tuple of a microservice m at time t as $\mathbf{x}_{m,t} = \langle x_{m,t}^w, x_{m,t}^1, \dots, x_{m,t}^H \rangle \in \mathbb{R}^{H+1}$, where $x_{m,t}^w$ is the workload and the rests are H system states.

Since indicators may vary over different time steps, we typically record a series of historical data to observe the changes in the microservice. Therefore, we have Def. 3.

DEFINITION 3 (MICROSERVICE INDICATOR SERIES). A microservice m 's indicator series is $X_m = \{\mathbf{x}_{m,0}, \dots, \mathbf{x}_{m,T}\} \in \mathbb{R}^{T \times (H+1)}$, with T time steps. The workload series and system state series can be expressed as $X_m^w \in \mathbb{R}^{T \times 1}$ and $\{X_m^1, \dots, X_m^H\} \in \mathbb{R}^{T \times H}$ respectively. Correspondingly, the workload series and indicator series of M microservices in the LDC can be expressed as $X^w \in \mathbb{R}^{M \times T \times 1}$ and $\mathbf{X} \in \mathbb{R}^{M \times T \times (H+1)}$.

Based on the microservice information and indicators series, the formal definition of **Microservice Indicators Forecasting (MIF)** problem is given as Def. 4.

DEFINITION 4 (MIF PROBLEM). For M microservices $\{m\}$ in an LDC, given the information group $\{f, dc, cg, rz\}$ and their historical indicator series $\mathbf{X}$ in the past T time steps, the goal of **MIF** is to forecast the indicators $\tilde{\mathbf{X}} \in \mathbb{R}^{M \times 1 \times (H+1)}$ at time $T + 1$.

2.4 Macro Graph v.s. Micro Matrix

After careful observation, we notice that the dependencies among microservices and the relationships between the monitoring indicators can significantly affect the results of the MIF problem. Therefore, in this subsection, we will characterize the relationships between microservices and their intrinsic indicators from both macro and micro perspectives.

Macro Graph. The relationships between different microservices are highly relevant to analyzing their operational status. According to the LDC hierarchy, there are three types of relationships in a top-down manner:

- **Collaboration Edges between Data Centers.** Microservices that are split across different Data Centers by Spanner should perform the same functionality, which is similar to each other. We can form *collaboration edges* between them (as a clique). An example collaboration edge is the one connecting RZ42A and RZ43A in Fig. 3.
- **Backup Edges between Cell Groups.** A pair of replica microservices within the same Cell Group should also have similar operational logic, allowing the establishment of *backup*

edges. An example is the dotted line between RZ42A and RZ42B in Fig. 3.

- **Correlation Edges between Region Zones.** Microservices working within Region Zones may have some hidden relationships, resulting in *correlation edges*. For example, as described in Fig. 2, since the Cashier microservice invokes the Trade microservice, their indicator series share similar patterns, with the former preceding the latter in time steps, resembling two similar yet staggered curves on a timeline (as the dashed line within RZ42A).

Correspondingly, we can form a heterogeneous Macro Graph to explore the relationships between microservices, enabling more accurate forecasting of indicator series, as shown in Def. 5.

DEFINITION 5 (MACRO GRAPH). A Macro Graph $G = (V, E, w)$ consists of M microservices m , each as a vertex v_m , and three types of weighted edges $E = E_c \cup E_b \cup E_{cr}$. Collaboration edges (E_c) connect vertices that are split by a spanner for the same functionality, backup edges (E_b) connect pairwise replica vertices, while correlation edges (E_{cr}) connect vertices with hidden, similar working patterns. Each edge $e \in E$ is associated with a weight $w(e) \in \mathbb{R}$ that quantifies the strength or intensity of the corresponding relationship.

Micro Matrix. We are also interested in the inner relationship between indicators. Hence, we randomly pick a microservice m and implement the causal analysis [15] on its indicator series X_m , as shown in Fig. 4. This graph plots historical data over 400 time steps, with an interval of 1 minute. The Y axis exhibits normalized values between 0 and 1 for four categories: Workload, CPU usage (CU), Memory Usage (MU), and Response Time (RT). To better illustrate the trend, we draw a plot line highlighting the moving average value using a 10-minutes time window.

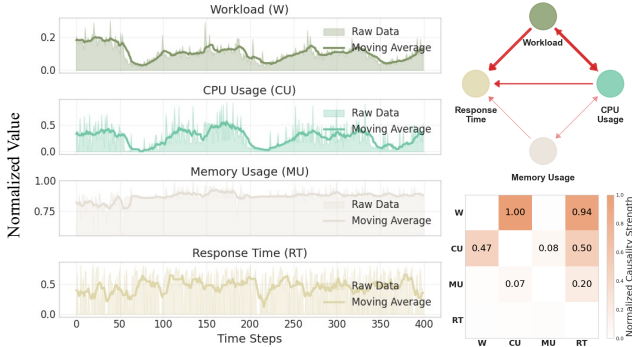


Figure 4: Causal Analysis of Microservice Indicators. 4 indicators (left) and their causal pattern with strength (right).

From Fig. 4, we observe that the Workload first affects CPU usage. Subsequently, both the workload and CPU usage jointly propagate their impact on the service quality like Response Time, showing an obvious causal relationship. Whereas Memory Usage is relatively independent, which is not directly affected by workload. Since memory is a static resource, it usually fluctuates within a relatively stable range during the execution of a microservice.

Correspondingly, we can quantify the inter-indicators causality of a microservice using a causal strength matrix (referred to as

a Micro Matrix), which is computed from historical data. Later, a group of matrices encapsulating the causal information from M microservices will be involved in the forecasting computation for the MIF problem. The definition of *Micro Matrix* is shown in Def. 6.

DEFINITION 6. For a microservice m , define A_m as its micro matrix representing the causal strength between its pairwise indicators (usually computed from historical data), and define $A = \{A_1, \dots, A_M\}$ as a Micro Matrix Group to record causal strength values results from M microservice.

3 MaMiClif: A MIF Framework

The proposed framework MaMiClif is illustrated in Fig. 5, which consists of three components: (1) Preprocessing: constructing Macro Graph G and computing Micro Matrix Group A . (2) Heterogeneous Graph Convolution: capturing the macro dependencies among microservices with G . (3) Causal Sparse Self-Attention: extracting the representations of indicator series with the support of inter-indicator causality in A .

3.1 Preprocessing

3.1.1 Macro Graph Construction. To compute Macro Graph $G = (V, E, w)$, we treat the M microservices as a node set $V = \{m\}$, and construct heterogeneous edge sets $E = E_c \cup E_b \cup E_{cr}$ from the Architecture Level and Correlation Level to distinctly model these macro spatial dependencies. The pseudo code is shown in Alg. 1.

Algorithm 1: Macro Graph Construction

Input: Microservice node set $V = \{m\}$ with ID labels $\{f, dc, cg, rz\}$, workload series X^w , similarity threshold γ .

Output: Macro Graph $G = (V, E, w)$

```

1 forall microservices  $m, n$  do
2   // Architecture Level Modeling
3   if  $f_m = f_n \wedge dc_m \neq dc_n$  then
4     Add  $(v_m, v_n)$  into  $E_c$  as a collaboration edge;
5     Set  $w(v_m, v_n) \leftarrow 1$ ;
6   else if  $f_m = f_n \wedge cg_m = cg_n \wedge rz_m \neq rz_n$  then
7     Add  $(v_m, v_n)$  into  $E_b$  as a backup edge;
8     Set  $w(v_m, v_n) \leftarrow 1$ ;
9   // Correlation Level Modeling
10  if  $PCC(X_m^w, X_n^w) > \gamma$  then
11    Add  $(v_m, v_n)$  into  $E_{cr}$  as a correlation edge;
12    Set  $w(v_m, v_n) \leftarrow PCC(X_m^w, X_n^w)$ ;
13 return Macro Graph  $G$ 

```

Here, E_c and E_b depict the static characteristics between microservices. Both edge types are assigned a uniform weight of 1, as the mere existence of these relationships is more critical than a continuous strength measure. Specifically, E_c connects microservices with the same functionality, but deployed in different data centers, to characterize the collaborative relationship (Lines 2-4); while E_b connects microservices with the same functionality, deployed in

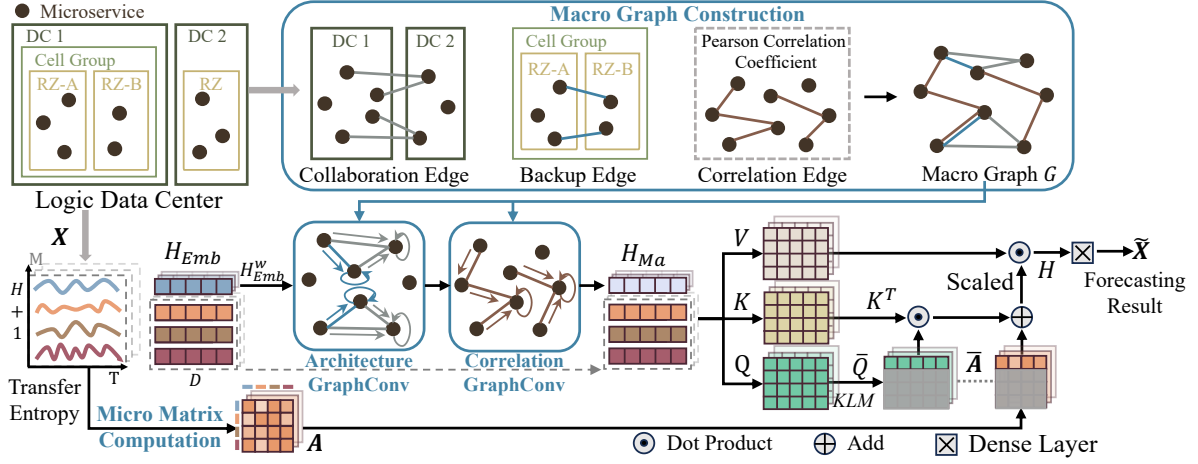


Figure 5: MaMiClif Architecture. Macro Graph and Micro Matrix model macro–micro dependencies in LDC; workload dependencies are captured via heterogeneous graph convolution, and indicator causality via Causal Sparse Self-Attention.

the same cell group, but in different region zones, to model the backup relationship (Lines 5-7).

On the other hand, E_{cr} depicts the dynamically changing relationships between microservices over T time steps. We employ the Pearson Correlation Coefficient (PCC), which is a widely adopted series similarity measurement [2], to quantify these correlations. Specifically, for two microservices m and n with workload series X_m^w and X_n^w , the PCC is computed as:

$$\text{PCC}(X_m^w, X_n^w) = \frac{\sum_{t=1}^T (x_{m,t}^w - \bar{x}_m^w)(x_{n,t}^w - \bar{x}_n^w)}{\sqrt{\sum_{t=1}^T (x_{m,t}^w - \bar{x}_m^w)^2} \sqrt{\sum_{t=1}^T (x_{n,t}^w - \bar{x}_n^w)^2}}$$

where $\bar{x}_m^w$ and $\bar{x}_n^w$ denote the sample means of X_m^w and X_n^w . An edge is added to E_{cr} only if this PCC exceeds a threshold γ , ensuring that only statistically significant correlations are retained (Lines 8-10).

The construction of G runs in $O(M^2T)$ time, dominated by pairwise PCC computations over M microservices with T -length series.

3.1.2 Micro Matrix Group Computation. Due to the advantages of non-linear, directed, and dynamic scalability [41], Transfer Entropy (TE) [31] is adopted to quantify the causal strength of indicators. Given m 's indicator series $X_m^i, X_m^j \in \{X_m^w, X_m^1, \dots, X_m^H\}$, TE quantifies the additional gain in predicting the future $x_{m,t+1}^j$ when the historical information of both X_m^i and X_m^j is provided, compared to using only the X_m^j history. Based on this, the causal strength of X_m^i on X_m^j is calculated as:

$$TE_{X_m^i \rightarrow X_m^j}^l = \sum_t p(x_{m,t+1}^j, x_{m,[t-l:t]}^j, x_{m,[t-l:t]}^i) \log \frac{p(x_{m,t+1}^j | x_{m,[t-l:t]}^j, x_{m,[t-l:t]}^i)}{p(x_{m,t+1}^j | x_{m,[t-l:t]}^j)} \quad (1)$$

where $p(\cdot)$ is the probability. l is the causality look-back window. The history of X_m^i is written as $x_{m,[t-l:t]}^i$, which is the shorthand

notation for $\langle x_{m,t-l}^i, x_{m,t-l+1}^i, \dots, x_{m,t}^i \rangle$. Eqn. (1) traverses each available time point t , using the joint probability as weights and performs a weighted summation of the mutual information difference. The joint probability is realized by a sum over all available realizations of $(x_{m,t+1}^j, x_{m,[t-l:t]}^j, x_{m,[t-l:t]}^i)$ in the indicator series.

After obtaining the TE between each pair of indicator series of m , we obtain the non-symmetric causal strength matrix $A \in \mathbb{R}^{(H+1) \times (H+1)}$, and further get the causal strength matrix group $\mathbf{A} \in \mathbb{R}^{M \times (H+1) \times (H+1)}$ of all microservices.

The computation of $\mathbf{A}$ involves estimating TE for all ordered pairs of $(H+1)$ indicators across M microservices. With time series length T , the total time complexity is $O(M(H+1)^2T)$, which is feasible given the small number of indicator types ($H+1 \ll M, T$).

3.2 Heterogenous Graph Convolution

After modeling the complex relationships of microservices with Macro Graph G , Indicator-Centric Embedding and Heterogeneous Graph Convolution are used to capture the microservice workload dependencies in a distinctive manner.

3.2.1 Indicator-Centric Embedding. Each indicator series exhibits unique characteristics. Therefore, to cope with this diversity and leverage the indicator-centric patterns, we employ inverted embedding [23] to encode each indicator series independently.

The difference between ordinary embedding and inverted embedding lies in their treatment of features. Ordinary embedding [35] projects the $(H+1)$ indicators to the embedding space, generating embeddings for each time stamp. However, this approach results in the entanglement of indicator features. In contrast, inverted embedding transposes the series to encode the time dimension T , forming embeddings for each indicator. The decoupling of indicator features can facilitate the clear capturing of indicator dependencies. The indicators embedding $H_{Emb} \in \mathbb{R}^{M \times (H+1) \times D}$ is calculated as:

$$H_{Emb} = \text{Embedding}(X^i)$$

where D is the embedding dimension, $\mathbf{X}^i \in \mathbb{R}^{M \times (H+1) \times T}$ is the inverted vector of the original indicator series $\mathbf{X}$.

3.2.2 Heterogeneous Graph Convolution. To capture the heterogeneous relationships among microservice nodes V , relational graph convolution [30] is used to learn the spatial dependencies of workloads based on Macro Graph. The hidden states is updated as:

$$h_m^{u+1} = \sigma \left(\sum_{e \in E} \sum_{n \in N_m^e} \frac{1}{|N_m^e|} W_e^u h_n^u + W_0^u h_m^u \right)$$

where $h_m^u \in \mathbb{R}^D$ is the hidden state of microservice m 's workload in the u -th layer of the network, h_m^0 is m 's workload embedding in H_{Emb} . $E = \{E_C, E_B, E_{Corr}\}$ is the relation set of the Macro Graph G . N_m^e denotes the neighbor set of v_m under relation type $e \in E$. σ is the element-wise activation function, W_e^u is the relation-wise weight matrix. W_0^u is the weight matrix for the self-loop connection, allowing the node to retain its own information during aggregation. After updating the workload representations in H_{Emb} , we get the macro representation $H_{Ma} \in \mathbb{R}^{M \times (H+1) \times D}$.

The heterogeneous graph convolution operates on the Macro Graph with M nodes and at most $O(M^2)$ edges. With U layers in graph convolution and embedding dimension D , its time complexity is $O(UM^2D)$ in the worst case, though it is typically $O(UMD)$ in practice due to the sparsity of architectural and correlation edges.

3.3 Causal Sparse Self-Attention

We propose Causal Sparse Self-Attention, which integrates causal strength into the self-attention, enabling a comprehensive extraction of representations of indicator series.

The self-attention mechanism learns the indicator dependencies by calculating attention scores. To improve efficiency, Query Sparsity Measurement (KLM) [44] is employed to distinguish the relative "importance" of queries, thereby reducing complexity.

After linear projecting representations H_{Ma} to obtain Query, Key and Value, $K, Q, V \in \mathbb{R}^{M \times (H+1) \times D}$, the Kullback-Leibler divergence-based function $KLM(q_i, K)$ is used to measure the importance of the i -th indicator, i.e., q_i , to the attention score QK^T :

$$KLM(q_i, K) = \ln \sum_{j=1}^{H+1} e^{\frac{q_i k_j^T}{\sqrt{D}}} - \frac{1}{H+1} \sum_{j=1}^{H+1} \frac{q_i k_j^T}{\sqrt{D}}$$

The large $KLM(q_i, K)$ indicates that q_i plays a dominant role in the dot product calculation. Only the top $\ln(H+1)$ important q_i s are retained to calculate the attention score. We incorporate causal strength into the attention score when computing the attention weight matrix, which is calculated as:

$$CSAtt(K, Q, V, A) = softmax\left(\frac{\bar{Q}K^T + \bar{A}}{\sqrt{D}}\right)V \quad (2)$$

where $\bar{Q}$ is a sparse vector of Q that only contains the top $\ln(H+1)$ important queries. Similarly, $\bar{A}$ is a sparse vector of A that retains the same positions as $\bar{Q}$. $CSAtt$ achieves a time complexity of $O(M(H+1)\ln(H+1)D)$.

The final indicator representation $H \in \mathbb{R}^{M \times (H+1) \times D}$ is obtained after Causal Sparse Self-Attention. We utilize dense layers to transform H and obtain the forecasting result $\tilde{\mathbf{X}}$. Mean Square Error

Loss (MSELoss) is used to optimize the model. The formulations of MSELoss is provided in Appx. D.3.

4 Complexity and Scalability Analysis

In summary, MaMiClif has an $O(M^2T)$ preprocessing cost, dominated by the dynamic construction of correlation edges E_{cr} via pairwise workload calculation. In contrast, collaboration edges E_c and backup edges E_b are determined by system architecture, which are static and need not be recomputed unless the topology changes. The core model components (Heterogeneous Graph Convolution and Causal Sparse Self-Attention) scale linearly with M , ensuring good scalability for real-world LDCs. Table 1 summarizes the time and space complexity of each component in MaMiClif.

The bottleneck is not the deep learning components (Graph Convolution or Self-Attention), but the preprocessing stage, specifically the exhaustive pairwise correlation analysis. This is common in graph-based time-series modeling.

Table 1: Complexity of MaMiClif components. G: Macro Graph Construction; A: Micro Matrix Group Computation; CSS: Causal Sparse Self-Attention; HGC: Heterogeneous Graph Convolution.

Component	Time Complexity	Space Complexity
G	$O(M^2T)$	$O(MT(H+1) + M^2)$
A	$O(M(H+1)^2T)$	$O(MT(H+1) + M(H+1)^2)$
HGC	$O(UM^2D)$ (worst) $O(UMD)$ (practical)	$O(MD + M^2)$
CSS	$O(M(H+1)^2D)$	$O(M(H+1)D)$
Overall	$O(M^2T)$	$O(MT(H+1) + M^2)$

Remarks on Causal Sparse Self-Attention. The time complexity of Causal Sparse Self-Attention is dominated by the Query Sparsity Measurement (KLM) step, which requires computing full pairwise attention scores to evaluate the entropy-based importance of each query. This results in a per-microservice cost of $O((H+1)^2D)$, leading to the total $O(M(H+1)^2D)$ complexity reported in the table. Although the subsequent attention computation is sparsified—retaining only the top- $\lceil \ln(H+1) \rceil$ queries and thus costing only $O(M(H+1)\ln(H+1)D)$ —the KLM scoring remains the computational bottleneck. Therefore, the overall complexity reflects the full pairwise scoring phase, consistent with the original Sparse Self-Attention in Informer [44].

5 Experiments

5.1 Experiment Setup

5.1.1 Datasets. Two microservice indicator datasets are used to validate the performance of MaMiClif:

LDC_MS is the exclusive dataset that records the 100 microservices' indicator series in an LDC over one week. The temporal granularity is 10 minutes, resulting in a total of 1,284 time steps. For each microservice, 4 key indicators are collected: Number of Requests (workload), CPU Usage, Memory Usage, and Response Time. The LDC has 2 Data Centers, 4 Cell Groups, 8 Regional Zones, and the microservices correspond to 13 functionalities. Based on

Table 2: Comparison results with different look-back window L on two datasets. The best results are in boldface.

Datasets	Model	MAE ↓				MSE ↓			
		$L = 72$	$L = 144$	$L = 288$	Avg.	$L = 72$	$L = 144$	$L = 288$	Avg.
LDC_MS	DeepAR	0.1621	0.2167	0.2470	0.2086	0.3084	0.8716	1.1938	0.7913
	TimesNet	0.1259	0.1512	0.1887	0.1553	0.2799	0.4057	0.4491	0.3782
	Pyraformer	0.1298	0.1372	0.1700	0.1457	0.1810	0.2443	0.3172	0.2475
	DLinear	0.1738	0.1565	0.1574	0.1626	0.1935	0.2511	0.2997	0.2481
	PatchTST	0.0456	0.0757	0.0852	0.0688	0.0194	0.1099	0.1635	0.0976
	Crossformer	0.1339	0.1549	0.2603	0.1830	0.2688	0.3394	0.4505	0.3529
	TSMixer	0.1510	0.1571	0.1745	0.1609	0.1464	0.1378	0.1701	0.1514
	iTransformer	0.0476	0.0617	0.0816	0.0636	0.0199	0.0310	0.0525	0.0345
	STSSL	0.1342	0.1633	0.1878	0.1617	0.1521	0.1726	0.1900	0.1715
	STEAM	0.1047	0.1156	0.1395	0.1199	0.1323	0.1948	0.2103	0.1791
	MaMiClif	0.0439	0.0565	0.0710	0.0571	0.0129	0.0282	0.0393	0.0268
Fisher	DeepAR	0.1285	0.1279	0.1241	0.1268	0.0603	0.0695	0.0570	0.0623
	TimesNet	0.1261	0.1267	0.1219	0.1249	0.0491	0.0484	0.0470	0.0481
	Pyraformer	0.1231	0.1246	0.1286	0.1254	0.0481	0.0479	0.0492	0.0484
	DLinear	0.1272	0.1300	0.1239	0.1270	0.0475	0.0483	0.0473	0.0477
	PatchTST	0.1159	0.1155	0.1151	0.1155	0.0456	0.0453	0.0451	0.0453
	Crossformer	0.1501	0.1450	0.1495	0.1482	0.0569	0.0558	0.0580	0.0569
	TSMixer	0.1850	0.1838	0.1846	0.1845	0.1222	0.1031	0.0741	0.0998
	iTransformer	0.1166	0.1180	0.1191	0.1179	0.0468	0.0469	0.0471	0.0469
	STSSL	0.1534	0.1725	0.1545	0.1601	0.0570	0.0647	0.0652	0.0623
	STEAM	0.1020	0.1110	0.1218	0.1116	0.0454	0.0498	0.0469	0.0473
	MaMiClif	0.0993	0.0974	0.1012	0.0993	0.0457	0.0432	0.0442	0.0443

the system architecture, we construct a Macro Graph containing 290 collaboration edges and 49 backup edges.

Fisher² contains the indicator series of 10 containers within a Kubernetes framework over a 30-day period. For each container, 11 indicators are recorded: Workload, CPU Usage, Time Wait, etc. We standardize the time step to 10 minutes using max-pooling. Since open-source datasets rarely include system architecture information, only correlation relationships are used to construct G .

5.1.2 Baselines. We use several state-of-the-art models for comparative experiments, including temporal forecasting methods:

DeepAR [8], a method involves recurrent neural networks to project prediction into a Gaussian distribution. TimesNet [38], a 2D convolution-based forecasting method. Pyraformer [22], a multi-scale forecasting method based on pyramid self-attention. DLinear [42] utilize linear layers to capture temporal correlation for multivariate time series forecasting. PatchTST [28] adopts channel-independence transformer with patching operation. Crossformer [43] employs Dimension Segment Wise embedding to capture cross-time and cross-dimension dependencies. TSMixer [10], an MLP-based method for multivariate forecasting. iTransformer [23], an indicator-centric Transformer framework with inverted embedding.

Spatial-temporal forecasting methods: STSSL [18], a self-supervised spatial-temporal method based on adaptive graph augmentation. STEAM [13] uses cross-view contrastive learning to extract robust spatial-temporal features of microservice workload series.

5.1.3 Evaluation Metrics. We use two widely adopted metrics in series forecasting, Mean Absolute Error (MAE) and Mean Squared Error (MSE) (details in Appx. D.). MAE measures the mean absolute value of all errors and is less sensitive to large errors. MSE places more emphasis on the large errors through the squaring calculation.

5.1.4 Parameter Settings. We apply min-max normalization with a 7:1:2 training-validation-test split. All models are trained on an NVIDIA V100 GPU with the Adam optimizer. The batch size is 128, The representation dimension D is 512. The learning rate is 0.001. The number of graph convolution layers U is 2. The causality look-back window size l is set as 8. The similarity threshold γ is set to the value of top 10% Pearson Correlation Coefficients.

5.2 Comparative Results

Table 2 shows the comparative results between MaMiClif and baselines. The following observations can be made:

MaMiClif outperforms temporal and spatial-temporal forecasting methods on **MIF** problem in LDC. Temporal methods cannot capture the microservice dependencies from a macro perspective. From a micro perspective, they capture the temporal dependencies using techniques like convolutional neural networks and self-attention mechanisms, but ignore the causal relationships between indicators. Notably, methods that employ inverted embedding, such as TSMixer, iTransformer, and MaMiClif, exhibit relatively smaller MSE values in LDC_MS. This indicates fewer large errors in the forecasting, making them more suitable for resource estimation aimed

²<https://github.com/chrisliu1995/Fisher-model/tree/master>

at allocating cloud resources, demonstrating the effectiveness of Indicator-Centric Embedding.

In spatio-temporal methods, STSSL uses contrastive learning to extract dependencies from topological graphs. It cannot exploit dynamic correlations and is more suitable for topology-dominated scenarios, such as traffic flow. STEAM is designed for microservice indicator forecasting. It considers non-stationarity in the temporal dimension and captures microservices correlations in the spatial dimension. However, it lacks fine-grained modeling of the LDC hierarchy, resulting in weaker performance in LDC scenarios.

5.3 Ablation Study

We conduct ablation studies to validate key designs in MaMiClif on LDC_MS dataset, with the input length $L = 72$. Shown in Fig. 6.

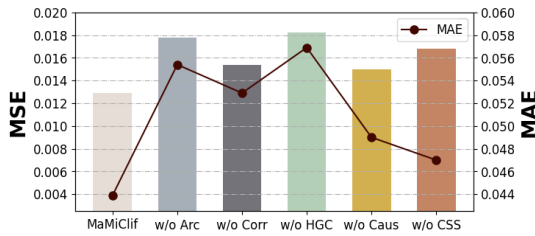


Figure 6: Ablation results of MaMiClif variants.

5.3.1 Effect of Macro Graph and Heterogeneous Graph Convolution. We ablate key components of the Macro Graph and its processing. w/o Arc removes architecture-level edges (E_c, E_b), using only correlation edges E_{cr} . The significant performance drop demonstrates that system topology is essential for modeling workload dependencies. w/o Corr removes correlation edges E_{cr} , causing a mild decline and validating the value of time-series similarity in capturing dynamic interactions. w/o HGC removes Heterogeneous Graph Convolution entirely, ignoring all macro dependencies, and yields the worst performance. The performance degradation highlights the necessity of graph-based modeling in MIF.

5.3.2 Effect of Causal Strength Matrix and Causal Sparse Self-Attention. The w/o Caus variant removes the causal strength matrix from Eqn. (2) and replaces $CSAtt$ with sparse self-attention [44]. The performance drop confirms that explicitly incorporating causal relationships enhances indicator representation learning.

The w/o CSS variant uses vanilla self-attention [23] instead of our causal sparse design. Despite the degradation, it still outperforms iTransformer, as it benefits from Heterogeneous Graph Convolution to capture macro-level microservice dependencies—highlighting the importance of spatial modeling beyond intra-series attention.

5.4 Parameter Sensitivity

We investigate the sensitivity of MaMiClif to the embedding dimension D and similarity threshold γ on LDC_MS dataset.

5.4.1 Embedding Dimension. As shown in Fig. 7, the optimal D grows with the look-back window L : $D = 256$ for $L = 72$, 512 for $L = 144$, and near 1024 for $L = 288$. Too small D limits representational capacity, while too large D causes feature sparsity. This

characteristic is rooted in the Indicator-Centric Embedding, which projects the temporal dimension into a D -dimension. Longer series contain more information and require a larger D for adequate representation, which differs from ordinary embedding schemes.

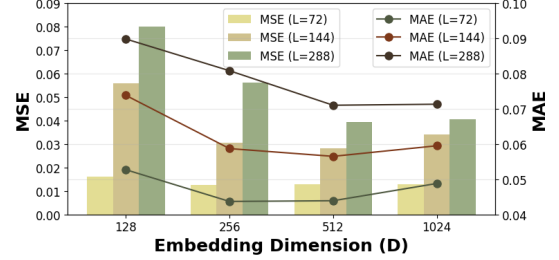


Figure 7: Performance w.r.t. D .

5.4.2 Similarity Threshold. As shown in Fig. 8, we vary the proportion of top PCC values used to construct correlation edges E_{cr} . With 100 microservices, there are $\binom{100}{2} = 4,950$ unique pairs; retaining the top 1%, 5%, 10%, and 20% corresponds to 50, 248, 495, and 990 edges, respectively. The top 5% yields optimal performance: a higher threshold introduces spurious correlations due to excessive graph density, while a lower one (e.g., 1%) discards meaningful dependencies. Thus, 5% best balances sparsity and signal fidelity.

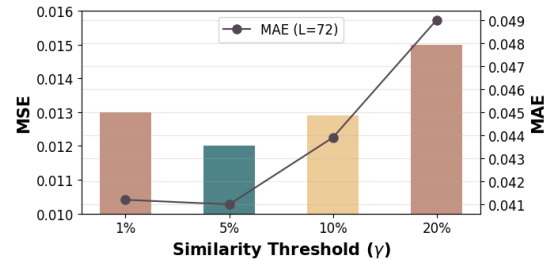


Figure 8: Performance w.r.t. γ with the input length $L = 72$.

6 Conclusion

In this paper, we propose MaMiClif for microservice indicators forecasting in Logical Data Centers (LDCs). MaMiClif models macro-level spatial dependencies among microservices via a Macro Graph, while capturing micro-level causal relationships between indicators through a Micro Matrix. These two perspectives are jointly exploited by Heterogeneous Graph Convolution and Causal Sparse Self-Attention to learn comprehensive spatio-temporal representations, thereby enhancing forecasting accuracy. Experiments on two real-world datasets validate the effectiveness of MaMiClif.

Acknowledgments

This work was supported by the National Key R&D Program of China [2024YFF0617700], the National Natural Science Foundation of China [U23A20309, 62372296, 62441236], and CCF-Ant Research Fund [CCF-AFSG RF20230408].

References

- [1] Muhammad Abdullah, Waheed Iqbal, Josep Lluís Berral, Jorda Polo, and David Carrera. 2022. Burst-Aware Predictive Autoscaling for Containerized Microservices. *IEEE Trans. Serv. Comput.* (2022), 1448–1460.
- [2] J. Benesty, J. Chen, Y. Huang, and I. Cohen. 2009. Pearson Correlation Coefficient. *Noise Reduction in Speech Processing* (2009), 1–4.
- [3] Rodrigo N. Calheiros, Enayat Masoumi, Rajiv Ranjan, and Rajkumar Buyya. 2015. Workload Prediction Using ARIMA Model and Its Impact on Cloud Applications' QoS. *IEEE Trans. Cloud Comput.* (2015), 449–458.
- [4] Jiadong Chen, Yang Luo, Xiuqi Huang, Fuxin Jiang, Yangguang Shi, Tieying Zhang, and Xiaofeng Gao. 2023. IPOC: An Adaptive Interval Prediction Model based on Online Chasing and Conformal Inference for Large-Scale Systems. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 202–212.
- [5] Mincheng Chen, Jingling Yuan, Dongling Liu, and Tao Li. 2020. An adaption scheduling based on dynamic weighted random forests for load demand forecasting. *The Journal of Supercomputing* 76, 3 (2020), 1735–1753.
- [6] Ke Cheng, Sheng Zhang, Chenghong Tu, Xiaohang Shi, Zhaocheng Yin, Sanglu Lu, Yu Liang, and Qing Gu. 2023. Proscale: Proactive autoscaling for microservice with time-varying workload at the edge. *IEEE Transactions on Parallel and Distributed Systems* (2023), 1294–1312.
- [7] Ka Ho Chow, Umesh Deshpande, Sangeetha Seshadri, and Ling Liu. 2022. DeepRest: deep resource estimation for interactive microservices. In *European Conference on Computer Systems, EuroSys*. 181–198.
- [8] Salinas David, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. 2020. DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks. *International Journal of Forecasting* 36 (2020), 1181–1191.
- [9] Docker. 2013. <https://www.docker.com>.
- [10] Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. TSMixer: Lightweight MLP-Mixer Model for Multivariate Time Series Forecasting. In *Conference on Knowledge Discovery and Data Mining, KDD*. 459–469.
- [11] Binbin Feng and Zhijun Ding. 2023. GROUP: An End-to-end Multi-step-ahead Workload Prediction Approach Focusing on Workload Group Behavior. In *ACM The Web Conference (WWW)*. 3098–3108.
- [12] Yu Gan, Yanqi Zhang, Kelvin Hu, Dailun Cheng, Yuan He, Meghna Pancholi, and Christina Delimitrou. 2019. Seer: Leveraging Big Data to Navigate the Complexity of Performance Debugging in Cloud Microservices. In *International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS*. 19–33.
- [13] Mohan Gao, Kexin Xu, Xiaofeng Gao, Tengwei Cai, and Haoyuan Ge. 2025. Spatial-Temporal Heterogenous Graph Contrastive Learning for Microservice Workload Prediction. In *Conference on Artificial Intelligence, AAAI*. 11681–11689.
- [14] Google. 2021. <https://cloud.google.com/blog/topics/developers-practitioners/microservices-architecture-google-cloud>.
- [15] C. W. J. Granger. 1969. Investigating Causal Relations by Econometric Models and Cross-Spectral Methods. (1969), 424–438.
- [16] Qin Hua, Dingyu Yang, Shiyu Qian, Hanwen Hu, Jian Cao, and Guangtao Xue. 2023. KAE-Informer: A Knowledge Auto-Embedding Informer for Forecasting Long-Term Workloads of Microservices. In *ACM The Web Conference WWW*. 1551–1561.
- [17] Vinodh Kumaran Jayakumar, Jaewoo Lee, In Kee Kim, and Wei Wang. 2020. A Self-Optimized Generic Workload Prediction Framework for Cloud Computing. In *International Parallel and Distributed Processing Symposium (IPDPS)*. 779–788.
- [18] Jiahao Ji, Jingyuan Wang, Chao Huang, Junjie Wu, Boren Xu, Zhenhe Wu, Junbo Zhang, and Yu Zheng. 2023. Spatio-Temporal Self-Supervised Learning for Traffic Flow Prediction. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI*. 4356–4364.
- [19] Michael T. Krieger, Óscar Torreño Tirado, Oswaldo Trelles, and Dieter Kranzlmüller. 2017. Building an open source cloud environment with auto-scaling resources for executing bioinformatics and biomedical workflows. *Future Gener. Comput. Syst.* (2017), 329–340.
- [20] Kubernetes. 2015. <https://kubernetes.io>.
- [21] Zhonghang Li, Chao Huang, Lianghao Xia, Yong Xu, and Jian Pei. 2022. Spatial-Temporal Hypergraph Self-Supervised Learning for Crime Prediction. In *International Conference on Data Engineering, ICDE*. 2984–2996.
- [22] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiya Lin, Alex X. Liu, and Schahram Dustdar. 2022. Pyraformer: Low-Complexity Pyramidal Attention for Long-Range Time Series Modeling and Forecasting. In *The Tenth International Conference on Learning Representations, ICLR*.
- [23] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *International Conference on Learning Representations, ICLR*.
- [24] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, Guodong Yang, and Chengzhong Xu. 2023. Erms: Efficient Resource Management for Shared Microservices with SLA Guarantees. In *International Conference on Architectural Support for Programming Languages and Operating Systems ASPLOS*. 62–77.
- [25] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Guodong Yang, and Chengzhong Xu. 2022. The power of prediction: microservice auto scaling via workload learning. In *Proceedings of the 13th Symposium on Cloud Computing, SoCC*. 355–369.
- [26] Yang Luo, Mohan Gao, Zhemeng Yu, Haoyuan Ge, Xiaofeng Gao, Tengwei Cai, and Guihai Chen. 2024. Integrating System State into Spatio Temporal Graph Neural Network for Microservice Workload Prediction. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.
- [27] Yang Luo, Shiyu Wang, Zhemeng Yu, Wei Lu, Xiaofeng Gao, Lintao Ma, and Guihai Chen. 2024. Adaptive Two-Stage Cloud Resource Scaling via Hierarchical Multi-Indicator Forecasting and Bayesian Decision-Making. *CoRR* (2024).
- [28] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations, ICLR*.
- [29] Jinwoo Park, Byungkwon Choi, Chunghwan Lee, and Dongsu Han. 2021. GRAF: a graph neural network based proactive resource allocation framework for SLO-oriented microservices. In *CoNEXT '21: The 17th International Conference on emerging Networking EXperiments and Technologies*. 154–167.
- [30] Michael Sejr Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *The Semantic Web International Conference, ESWC*. 593–607.
- [31] T. Schreiber. 2000. Measuring information transfer. *Physical Review Letters* (2000), 461–464.
- [32] Da Sun Handason Tam, Yang Liu, Huanle Xu, Siyue Xie, and Wing Cheong Lau. 2023. PERT-GNN: Latency Prediction for Microservice-based Cloud-Native Applications via Graph Neural Networks. In *Conference on Knowledge Discovery and Data Mining, KDD*. 2155–2165.
- [33] Chanh Nguyen Le Tan, Cristian Klein, and Erik Elmroth. 2019. Multivariate LSTM-Based Location-Aware Workload Prediction for Edge Data Centers. In *International Symposium on Cluster, Cloud and Grid Computing, CCGRID*. 341–350.
- [34] Sean J. Taylor and Benjamin Letham. 2017. Forecasting at Scale. *PeerJ Prepr.* (2017), e3190.
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems, NIPS* 30 (2017).
- [36] Shiyu Wang. 2024. NeuralReconciler for Hierarchical Time Series Forecasting. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 731–739.
- [37] Xiao Wang, Nian Liu, Hui Han, and Chuan Shi. 2021. Self-supervised Heterogeneous Graph Neural Network with Co-contrastive Learning. In *ACM SIGKDD Conference on Knowledge Discovery & Data Mining, KDD*. 1726–1736.
- [38] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *The Eleventh International Conference on Learning Representations, ICLR*.
- [39] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojuan Chang, and Chengqi Zhang. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 753–763.
- [40] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *International Joint Conference on Artificial Intelligence IJCAI*. 3634–3640.
- [41] Zhemeng Yu, Yang Luo, Yucen Gao, Yinbo Sun, Xiaofeng Gao, Lintao Ma, and Guihai Chen. 2025. CausalScaler: A Causality-Driven Autoscaling Framework for the Cloud. In *International Conference on Database Systems for Advanced Applications DASFAA*.
- [42] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are Transformers Effective for Time Series Forecasting?. In *Conference on Artificial Intelligence, AAAI*. 11121–11128.
- [43] Yunhao Zhang and Junchi Yan. 2023. Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting. In *International Conference on Learning Representations, ICLR*.
- [44] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *Conference on Artificial Intelligence, AAAI*. 11106–11115.
- [45] Zhiqiang Zhou, Chaoli Zhang, Lingna Ma, Jing Gu, Huajie Qian, Qingsong Wen, Liang Sun, Peng Li, and Zhimin Tang. 2023. AHPA: Adaptive Horizontal Pod Autoscaling Systems on Alibaba Cloud Container Service for Kubernetes. In *Conference on Artificial Intelligence, AAAI*. 15621–15629.
- [46] Aaron Zou, Wei Lu, Zhibo Zhu, Xingyu Lu, Jun Zhou, Xiaojin Wang, Kangyu Liu, Kefan Wang, Renen Sun, and Haiqing Wang. 2024. OptScaler: A Collaborative Framework for Robust Autoscaling in the Cloud. *Proc. VLDB Endow.* 17, 12 (2024), 4090–4103.

A Microservice Updates

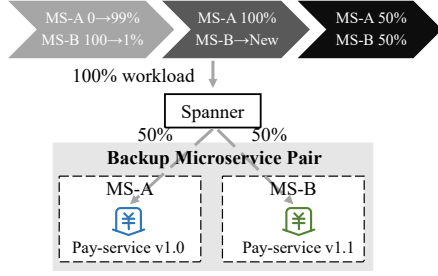


Figure 9: Microservice Blue-Green Deployment updates: workload shifts from MS-B (old) to MS-A (new); after stabilization, MS-B upgrades to maintain load balancing.

B Related Work

In this section, we will introduce two categories of microservice indicators forecasting methods, which differ mainly in whether they utilize the spatial dependencies of microservices.

B.1 Temporal Forecasting Methods

As mentioned, workload is crucial among microservice indicators [6, 25], and it's intuitive and reasonable to model workload as time series to forecast. Statistical methods [34] and machine learning methods [3, 5] have been applied to workload forecasting, offering high efficiency and interpretability. To address the dynamic nature of workloads and tackle concept drift, online ensemble learning [4] has been employed to dynamically select models according to the pattern of the data. However, these methods are designed for univariate time series, which cannot capture the causalities among variables and cannot utilize spatial dependencies.

With the development of the deep learning, recurrent neural networks-based algorithms have been used for workload forecasting [17, 33]. iTransformer [23] introduces inverted embedding in the Transformer framework, enabling better exploration of variables and demonstrating strong performance in workload forecasting. KAE-Informer [16] further integrates series decomposition with ProbSparse Self-attention Mechanism [44], achieves diverse load prediction while reducing computational complexity. Harmony [27] and CausalScaler [41] extend the task to multi-indicators. Harmony uses hierarchical attention [36] to comprehensively model the inter-indicator dependencies. CausalScaler adopts temporal causal learning to collaboratively learn the dependencies among indicators, enhancing forecasting robustness.

B.2 Spatial-Temporal Forecasting Methods

The mutual calls between microservices bring natural spatial dependencies of these components. The Microservice Call Graph (MCG) based methods [7, 29, 32] forecast indicators by leveraging the topological relationships generated by microservices calling behaviors. They fail to consider the non-invocation relationships among microservices, such as collaboration and backup ones, from a higher

system architecture level. The temporal features of microservice systems during execution are also neglected.

Spatio-Temporal Graph Neural Networks (STGNN) [39, 40] model spatial dependencies by treating microservices as nodes and constructing edges based on calling behavior [29, 32] or similarities. STAMP [26] constructs multi-graphs based on the series similarity and employs dynamic contrastive learning [21] to extract spatio-temporal features of workloads. To cope with the non-stationarity of workload series and the prior interference caused by the similarity threshold, STEAM [13] designs a spatiotemporal stacking structure with a de-stationary module in the long-term temporal feature encoding. It adopts heterogeneous graph contrastive learning [37] to enhance the forecasting robustness.

The STGNN-based methods simplify the complex relationships among microservices by treating them solely as calling- or similarity-based ones, without distinguishing or refining modeling for architecture and business dependencies. Additionally, there is a lack of exploration into multi-indicators, further limiting their applicability. However, these diverse graph construction methods and flexible network architectures inspire our design of the LDC microservice indicators forecasting model.

C Symbols and Notations

Table 2 provides a summary of the primary symbols in MaMiClif.

Table 3: Symbols and Notations

Symbols	Definitions
m, n	Microservice.
M	The number of microservices in the Logical Data Center.
H	The number of system states.
T	The number of time steps.
f_m	Microservice m 's functionality.
dc_m	Microservice m 's Data Center.
cg_m	Microservice m 's Cell Group.
rz_m	Microservice m 's Region Zone.
$x_{m,t}$	The indicator tuple of microservice m at time t .
$x_{m,t}^w$	The workload of microservice m at time t .
X_m	Microservice m 's indicator series.
X_m^w	The workload series of microservice m .
X^w	The workload series of M microservices.
$\mathbf{X}$	The indicator series of M microservices.
$\tilde{\mathbf{X}}$	The future indicators at time $T + 1$.
G	Macro Graph with node set V , edge set E and edge weight w .
E_c	Collaboration edges.
E_b	Backup edges.
E_{cr}	Correlation edges.
γ	Similarity threshold of correlation edges.
U	Number of layers in Heterogenous Graph Convolution.
$w(e)$	Edge weight of edge $e \in E$.
A_m	Micro Matrix of microservice m 's indicators.
$\mathbf{A}$	Micro Matrix of M microservices.

D Evaluation Metrics and Loss Function

D.1 Mean Squared Error (MSE).

Given ground-truth values $\mathbf{y} = \{y_1, \dots, y_N\}$ and predictions $\hat{\mathbf{y}} = \{\hat{y}_1, \dots, \hat{y}_N\}$, the Mean Squared Error is computed as:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

MSE is sensitive to outliers because large prediction errors (e.g., when $|y_i - \hat{y}_i| > 1$) are squared, which amplifies their contribution to the overall loss. However, this property also allows MSE to reflect the dispersion and variance of prediction errors, providing insight into the error distribution.

D.2 Mean Absolute Error (MAE).

The Mean Absolute Error is defined as the average absolute difference between predictions and ground-truth values:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|.$$

MAE is robust to outliers since it treats all errors linearly, without squaring. However, this also means it does not capture the variability or distributional characteristics of the prediction errors—only their average magnitude.

D.3 Mean Squared Error Loss

The Mean Squared Error (MSE) Loss is commonly used as the training objective in regression tasks. Given ground-truth targets $\mathbf{y} = \{y_1, \dots, y_N\}$ and model predictions $\hat{\mathbf{y}} = \{\hat{y}_1, \dots, \hat{y}_N\}$, the loss is defined as:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

This loss function penalizes larger errors more heavily due to the quadratic term, which encourages the model to avoid significant deviations during training.